SQL Project Case Study — Solved

Complete SQL answers for the 10 questions in the supplied case study.

Table used: orders(customer_id, order_date, item, quantity, price)

Note: The queries use SQLite-compatible date filtering for Question 2. For MySQL, the month filter can be written as MONTH(order_date) = 12.

1. List all items purchased by customer 10449.

```
SELECT item FROM orders WHERE customer_id = 10449 ORDER BY order_date;
```

Result / explanation: Items: Unicycle, Snow Shoes, Bicycle, Canoe, Flashlight, Canoe paddle.

2. Find the average price of items ordered in December.

```
SELECT ROUND(AVG(price), 2) AS average_price
FROM orders
WHERE strftime('%m', order_date) = '12';
```

Result / explanation: Average price: 105.75. (December prices: 22.00, 380.50, 280.00, 14.75.)

3. Find the lowest price among all tents.

```
SELECT MIN(price) AS lowest_tent_price
FROM orders
WHERE item = 'Tent';
```

Result / explanation: Lowest tent price: 79.99.

4. Show each item with its maximum and minimum recorded price.

```
SELECT item, MAX(price) AS maximum_price, MIN(price) AS minimum_price
FROM orders
GROUP BY item
ORDER BY item;
```

Result / explanation: Returns one row per item, with its maximum and minimum price. Items appearing once have the same maximum and minimum.

5. Show items whose maximum recorded price is greater than 190.00.

```
SELECT item, MAX(price) AS maximum_price
FROM orders
GROUP BY item
HAVING MAX(price) > 190.00
ORDER BY maximum_price DESC;
```

Result / explanation: Qualifying items: Parachute (1250.00), Bicycle (380.50), Canoe (280.00), Unicycle (192.50).

6. List items priced above 10.00, sorted by price in ascending order.

```
SELECT item, price
FROM orders
WHERE price > 10.00
ORDER BY price ASC;
```

Result / explanation: Returns all matching item/price rows from lowest to highest price.

7. Show customer ID, order date, and item, excluding Snow Shoes and Ear Muffs.

```
SELECT customer_id, order_date, item
FROM orders
WHERE item NOT IN ('Snow Shoes', 'Ear Muffs')
ORDER BY order_date;
```

Result / explanation: Returns all rows except those for Snow Shoes and Ear Muffs.

8. Show items and prices for items whose names start with S or P.

```
SELECT item, price
```

```
FROM orders
WHERE item LIKE 'S%' OR item LIKE 'P%'
ORDER BY item;
```

Result / explanation: Matching item names: Pogo stick, Parachute, Pillow, Pocket Knife, Ski Poles, Snow Shoes.

9. Show order date, item, and price for prices from 10.00 through 80.00.

```
SELECT order_date, item, price
FROM orders
WHERE price BETWEEN 10.00 AND 80.00
ORDER BY price ASC;
```

Result / explanation: BETWEEN includes both endpoints (10.00 and 80.00).

10. Calculate the per-unit price for each item.

```
SELECT item,
       ROUND(SUM(price) / SUM(quantity), 2) AS unit_price
FROM orders
GROUP BY item
ORDER BY unit_price DESC;
```

Result / explanation: Uses total recorded price divided by total quantity for each item. This follows the approach in the supplied case study.

Important corrections

• Question 1 asks for the items, so the answer selects item rather than every column. Use SELECT * if all order details are required.

• Question 2 filters the month using the date function, which is more reliable than matching a date string with LIKE.

• Question 8 says names start with S or P, so the correct condition is LIKE 'S%' OR LIKE 'P%'. The original condition searched for both letters anywhere in the name.

• Question 10 retains the supplied case study's calculation: SUM(price) / SUM(quantity), grouped by item.